

EventFlow / Your Events App

Enterprise Security & Architecture Audit

SOC 2 readiness and third-party penetration-test preparation

Field	Detail
Date	June 15, 2026
Scope	Full application (web + worker), production deployment at www.yourevents.app
Auditor role	Senior Software Architect / Security Engineer / DevOps / Code-Quality Auditor
Stack	Next.js 14 (App Router), Prisma + PostgreSQL, Stripe Connect, Redis, Railway
Basis	Current production code, verified June 2026; extends the prior June 2026 internal audit

Reader context. This application underwent a formal internal audit in June 2026 that remediated 4 Critical, 8 High, 14 Medium, and all Low findings (injection, RBAC, webhook integrity, oversell/concurrency, refund correctness). Those are confirmed closed and are not re-litigated here. This review assesses the current production state and gives particular scrutiny to the large authentication/account/payment-adjacent surface added recently (magic-link auth, attendee accounts, password reset, promo codes, refund-request flow, waitlist) — none of which had been through a formal review. Findings are therefore weighted toward enterprise/compliance readiness rather than basic vulnerability classes, most of which are already handled well.

1. Executive Summary

This is a well-architected, security-conscious codebase, materially stronger than a typical early-stage SaaS. App-layer security fundamentals are largely correct: queries are parameterized through Prisma, RBAC is enforced server-side at every dashboard and API surface, Stripe webhooks are signature-verified and idempotent, secret-bearing tokens are hashed at rest, and the recent auth work uses single-use short-TTL tokens with non-enumerable endpoints.

The gaps that matter are not classic vulnerabilities. They are compliance and operational-maturity gaps that a SOC 2 auditor and a penetration-test firm will flag immediately: no MFA, email-only sign-in to any role (including SUPERADMIN), no versioned database migrations in production, zero automated test coverage, no authentication-event audit logging, and no CI security scanning.

Score	Value	Rationale
Overall Health	78 / 100	Strong engineering, live and functional; held back by compliance/test/ops-maturity gaps
Security	80 / 100	App-layer controls strong; missing MFA, session revocation, auth audit logging, automated tests
Maintainability	74 / 100	Consistent patterns, good separation; zero tests, several oversized files, AI-verbose comments
Scalability	80 / 100	Stateless web + separate worker + Redis; single Postgres, cron-tick worker, no caching/read replicas
Performance	82 / 100	Bounded queries, parameterized analytics SQL, sensible includes; minor per-request query overhead

Production-ready for current scale: Yes, with conditions. SOC 2 / enterprise-ready: Not yet.

2. Critical & High-Risk Findings

There are no open Critical code-execution, injection, or broken-access-control vulnerabilities — the prior audit closed those and spot-checks confirm they remain fixed. The following High findings are enterprise/compliance-grade.

H-1 — Email-only authentication grants any role, including SUPERADMIN; no MFA

Risk: **High** · Category: Authentication / Privilege Escalation

Impact. /auth/verify signs an existing user in at their real role. Because magic-link and password-reset both rely solely on inbox control, compromise of a single email account equals full takeover of an Organizer or SUPERADMIN account — which controls payouts, refunds, attendee PII, and platform-wide admin. There is no second factor anywhere. For a platform moving money via Stripe Connect, this is the single highest-impact risk.

Evidence. src/app/auth/verify/route.ts (signSession with the found user's real role); no TOTP/WebAuthn implementation exists in src/.

Fix. Require MFA (TOTP or WebAuthn) for ADMIN/SUPERADMIN and ideally ORGANIZER. Restrict passwordless magic-link sign-in to ATTENDEE accounts only. Add step-up auth for money-movement actions (refunds, Connect changes). Maps to SOC 2 CC6.1.

H-2 — Production schema managed by prisma db push (no versioned migrations)

Risk: **High** · Category: Database / DevOps / Change Management

Impact. The boot command runs `prisma db push --skip-generate` and there is no prisma/migrations directory. This means no migration history, no reviewable or rollback-able schema-change record, and a real data-loss risk — db push silently alters columns, and a change it deems lossy fails the deploy at boot (already observed in production). SOC 2 CC8.1 requires auditable, reversible change management.

Evidence. package.json start script; confirmed absence of prisma/migrations.

Fix. Adopt prisma migrate — generate versioned migrations in development, commit them, deploy with `prisma migrate deploy` (not db push). Keep migration files in version control as the change-audit trail. Prerequisite for SOC 2.

H-3 — Zero automated test coverage on a payments + PII system

Risk: **High** · Category: Testing / Change Management

Impact. There are no unit, integration, or e2e tests (no test runner in devDependencies, no test files). The roadmap's own exit criterion (60% coverage on src/server/*) is unmet. For an app whose pricing, refund/fee math, atomic seat reservation, and RBAC are money-critical, every change ships unverified. SOC 2 treats this as a change-management control failure.

Evidence. No vitest/jest/playwright in package.json; no *.test.ts or __tests__.

Fix. Add Vitest for src/server/* (pricing, finance, tickets/seat reservation, refund math) and the auth libs (magic-link, password-reset), plus Playwright for the golden-path e2e. Hit critical-path 60% first, gate CI on it.

3. Detailed Findings

ID	Severity	Area	Description	Recommendation
H-1	High	AuthN / Priv-esc	Email-only sign-in to any role; no MFA (auth/verify/route.ts)	MFA for staff/admin; magic-link = attendee-only
H-2	High	DB / DevOps	prisma db push in prod, no versioned migrations (package.json)	Switch to prisma migrate deploy
H-3	High	Testing	No automated tests on payment/PII logic (whole repo)	Vitest on src/server/* + Playwright e2e

ID	Severity	Area	Description	Recommendation
M-1	Medium	Session mgmt	Password reset / credential change does not revoke existing JWT sessions (~7-day life) (lib/auth.ts)	Add tokenVersion to User; embed in JWT; bump on reset
M-2	Medium	Rate limiting	Limiter fails open on Redis outage — applies to magic-link/forgot/reset/signin (lib/rate-limit.ts)	Fail-closed (or local fallback) for auth/email endpoints
M-3	Medium	Logging / SOC 2	No audit logging of authentication events (login, magic-link, reset, role change)	Log authN events to AuditLog (actor, IP, UA, outcome)
M-4	Medium	Data integrity	Email case mismatch: signup/signin case-sensitive, magic-link lowercases -> duplicate users / sign-in failures	Lowercase-normalize email on all write/lookup paths
M-5	Medium	AuthN	No account lockout / credential-stuffing defense beyond per-IP rate limit (auth/signin)	Per-account failed-attempt lockout / backoff
M-6	Medium	AuthN	Weak password policy: min 8, no breach check, no complexity (signup/reset)	Raise floor; HavelBeenPwned k-anonymity check
M-7	Medium	Supply chain	No dependency scanning / SAST in CI (no Dependabot/Snyk/CodeQL)	Enable Dependabot + CodeQL; npm audit gate
M-8	Medium	Dependencies	Majors trailing: Next 14->15, Prisma 5->7, Stripe 16->latest, Sentry 8->9	Plan staged upgrades behind tests
L-1	Low	Container	Docker runs as root (no USER directive) (Dockerfile)	Add non-root USER node in runner stage
L-2	Low	Dead deps	bullmq installed but worker is a cron-tick; pino present but code uses console.* (package.json)	Remove unused deps or adopt them
L-3	Low	Crypto keys	One JWT_SECRET signs sessions and QR tickets (issuer-separated, fail-closed) (lib/auth.ts)	Separate keys per token purpose; plan rotation
L-4	Low	Observability	Sentry source maps not uploaded -> minified traces	Add SENTRY_AUTH_TOKEN + source-map upload
L-5	Low	Maintainability	Oversized multi-responsibility files (event dashboard page, lib/email.ts)	Extract sections into components/modules
L-6	Low	Data protection	Addresses/phone/email stored; confirm encryption-at-rest + retention	Document PG encryption; add retention/erasure policy
L-7	Low	Accepted risk	QR JWT never expires + token stored plaintext (single-use scan is the control)	Documented and accepted; revisit if QR re-issuance changes
L-8	Low	Open redirect (def-in-depth)	publicBase() falls back to Host header if NEXT_PUBLIC_APP_URL unset (set in prod, safe today)	Allowlist host; never trust Host for email/redirect URLs

Category notes

- Injection (SQLi/XSS/Cmd/Path): Strong. Prisma parameterizes; analytics raw SQL uses Prisma.sql / whitelisted tokens; email HTML escapes user input via esc(); CSV export neutralizes formula injection; no child_process/fs user-path usage; no binary file uploads. No findings.
- CSRF: Server Actions carry origin checks; session cookie is httpOnly + SameSite=Lax, blocking cross-site cookie-bearing POSTs. Adequate; document as a control.
- SSRF: No server-side fetch of user-supplied URLs found. No findings.
- IDOR: Spot-checked clean — refund-request and ticket retrieval require the per-row accessToken; account pages scope by session.sub; dashboard subpages use requireRole + orgScope; tokens are 24-byte random, hashed where reusable. No findings.
- Secrets: No hardcoded prod secrets; DEV_FALLBACK_SECRET is dev-only and fails closed in production. All real secrets are Railway env.
- Sensitive data in logs: Errors log e.message only; tokens/passwords/PII are not logged.

4. Quick Wins (under one day each)

1. Lowercase-normalize email everywhere (signup, signin, magic-link, forgot) — closes M-4. ~1-2 hrs.
2. Add USER node to the Dockerfile runner stage — closes L-1. ~30 min + redeploy test.
3. Enable GitHub Dependabot + CodeQL + npm audit CI gate — closes most of M-7. ~1-2 hrs.
4. Remove bullmq (and pino if unused) or wire them in — closes L-2. ~30 min.
5. Make auth/email rate-limiters fail closed (or in-memory fallback) — narrows M-2. ~2-3 hrs.
6. Add SENTRY_AUTH_TOKEN + source-map upload — closes L-4. ~1 hr.
7. Log authentication events to AuditLog (login, magic-link, reset) — starts M-3. ~3-4 hrs.
8. Raise password floor + HavelBeenPwned check on set/reset — closes M-6. ~3 hrs.

5. Technical Debt Report

High priority (compliance/operational blockers)

- No versioned migrations (H-2) — ~3-5 days to introduce prisma migrate, baseline the schema, switch the deploy command.
- No automated tests (H-3) — ~10-15 days to reach meaningful critical-path coverage (server logic + auth + golden-path e2e).
- No MFA / email-only privilege (H-1) — ~5-8 days for TOTP + enrollment + step-up + restricting magic-link to attendees.

Medium priority

- Session revocation / tokenVersion (M-1) — ~2 days.
- Auth-event audit logging (M-3) — ~2 days.
- Account lockout + email normalization + password policy (M-4/M-5/M-6) — ~3 days combined.
- Dependency-scanning CI + staged major upgrades (M-7/M-8) — ~4-6 days.

Low priority

- File decomposition (L-5), non-root container (L-1), key separation (L-3), source maps (L-4), unused deps (L-2), data-retention policy (L-6) — ~4-6 days combined.

Total estimated remediation: ~6-8 engineer-weeks to reach SOC 2 / pen-test readiness (one mid-to-senior engineer).

6. 30 / 60 / 90-Day Remediation Plan

Days 0-30 — Security & stability blockers

- H-2: introduce prisma migrate, baseline the prod schema, switch deploy to migrate deploy.
- H-1: MFA (TOTP) for ADMIN/SUPERADMIN/ORGANIZER; restrict magic-link to ATTENDEE; step-up on refunds/Connect.
- M-1, M-2, M-3: tokenVersion session revocation; fail-closed auth rate limiting; authentication-event audit logging.
- Quick wins 1-4, 6 (email normalization, non-root container, Dependabot/CodeQL, dep cleanup, source maps).

Days 31-60 — Test foundation & hardening

- H-3 part 1: Vitest on src/server/* (pricing, finance, tickets/seat reservation, refund math) + auth libs; CI gate.
- M-5, M-6: account lockout + password policy / breach check.
- M-8: staged dependency upgrades (Stripe, Sentry first; then Prisma 5->7, Next 14->15 behind tests).
- Data protection: document encryption-at-rest; write retention/erasure (GDPR / SOC 2 CC6.x) policy.

Days 61-90 — Architecture, performance, debt reduction

- H-3 part 2: Playwright golden-path + key role flows.
- Replace cron-tick worker with BullMQ (already a dep) for reliable, observable jobs.
- Add caching for public event/listing pages (ISR or Redis); consider read-replica routing for analytics SQL.
- Decompose oversized files (L-5); separate signing keys (L-3); finalize logout-everywhere UX.

7. Architecture, Performance & Quality

Architecture — Strengths. Clean Next.js App Router layout; clear separation (lib/, server/, route handlers, components); centralized cross-cutting logic (computeTotals for pricing, finance.ts for revenue, effectivePlan/eventEntitlements for gating, orgScope/requireRole for tenancy); separate worker service; consistently-enforced multi-tenant isolation.

Weaknesses. A few god-files (event dashboard page, email.ts), cron-tick worker instead of a real queue, no caching layer. Code-quality score: 7.5/10 — readable, consistently named, low duplication, well-commented (occasionally over-commented); main detractor is the absence of tests and a few high-complexity files.

Performance. Hot paths use bounded queries with appropriate includes; analytics use parameterized raw SQL with adaptive bucketing; atomic seat reservation is concurrency-proven. Minor overhead: the account layout adds 1-2 extra per-request queries; no public-page caching. No N+1 patterns observed in critical paths. No high-impact bottlenecks.

Database. Well-normalized with correct relations and indexes on all hot lookups (tokenHash unique, (eventId,status), email, FK indexes on new models). Primary DB finding is process, not schema: no migrations (H-2). Secondary: confirm encryption-at-rest + retention (L-6).

AI-generated code detection. The codebase is substantially AI-generated and shows it in uniformity — consistent naming, comment density, error-handling idioms, and security patterns. This is a strength for consistency but carries two risks: plausible-but-unverified logic ships without tests (reinforces H-3), and verbose comments can drift from code. The code does NOT show typical AI failure modes (copy-pasted insecure snippets, inconsistent auth, dead scaffolding) — it is notably disciplined. Mitigation is exactly H-3: tests to lock in behavior.

8. Final Verdict

Is this application production-ready?

- For its current stage (live SaaS, real payments): Yes — app-layer security fundamentals are sound and the prior audit's criticals/highs are closed.
- For SOC 2 certification and a clean third-party pen test: Not yet — the gaps are well-defined and remediable in ~6-8 engineer-weeks.

Overall risk assessment: Moderate. No open remote-exploitation vulnerability. Residual risk concentrates in account-takeover via email compromise of a high-privilege account (no MFA), operational change-management (no migrations, no tests), and compliance observability (no auth audit logging). None is actively bleeding, but each is what an auditor or pen-tester will lead with.

Top 10 recommendations (priority order)

9. Add MFA for staff/admin and restrict magic-link to attendees (H-1).
10. Switch production to versioned prisma migrate (H-2).
11. Build automated tests for pricing, refunds, seat-reservation, and auth (H-3).
12. Revoke sessions on credential change via tokenVersion (M-1).
13. Audit-log authentication events (M-3).
14. Make auth rate-limiting fail closed (M-2).
15. Normalize email case on all auth paths (M-4).
16. Add CI dependency/SAST scanning (M-7).
17. Account lockout + stronger password policy (M-5/M-6).
18. Non-root container, source-map upload, remove unused deps (L-1/L-4/L-2).

Estimated engineering effort to fully remediate: ~6-8 engineer-weeks (one mid-to-senior engineer), front-loaded with the three High findings in the first 30 days.